Exercise 06

Index No: 25000799
Registration No: 2025/CS/079

(01)

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <string.h>

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }

double int_div(double a, double b) {
    if (b == 0) {
        printf("Error occured : division by zero\n");
        return 0;
    }
    return (int)(a / b);
}

double norm_div(double a, double b) {
    if (b == 0) {
        printf("Error occured : division by zero\n");
        return 0;
    }
    return a / b;
}

double mod_op(double a, double b) {
    if (b == 0) {
        printf("Error occured : modulo by zero\n");
        return 0;
    }
    return (int)a % (int)b;
}

double power(double a, double b) { return pow(a, b); }

double gcd(double a, double b) {
    int x = (int)fabs(a), y = (int)fabs(b);
    while (y) {
        int t = y;
        y = x % y;
        x = t;
    }
    return x;
}

double lcm(double a, double b) {
    int x = (int)fabs(a), y = (int)fabs(b);
    if (x == 0 || y == 0) return 0;
    return (double)(x * y) / gcd(x, y);
}

double max_op(double a, double b) { return (a > b) ? a : b; }
double min_op(double a, double b) { return (a < b) ? a : b; }
double avg(double a, double b) { return (a + b) / 2.0; }
double absdiff(double a, double b) { return fabs(a - b); }
```

```
ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc que1.c

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out

MENU
 1. Add
 2. Subtract
 3. Multiply
 4. Divide (submenu)
 5. Modulus
 6. Power
 7. GCD
 8. LCM
 9. Max
10. Min
11. Average
12. AbsDiff
13. SumSq
14. ProdSq
15. Exit
16. History
Choice: 1
Enter two numbers: 14
15
Result: 29.00

MENU
 1. Add
 2. Subtract
 3. Multiply
 4. Divide (submenu)
 5. Modulus
 6. Power
 7. GCD
 8. LCM
 9. Max
10. Min
11. Average
12. AbsDiff
13. SumSq
14. ProdSq
15. Exit
16. History
Choice: 15
```

```
double sumsq(double a, double b) { return a * a + b * b; }
double prodsq(double a, double b) { return (a * a) * (b * b); }

typedef struct {
    char op[30];
    double a, b, result;
    int is_int;
} History;

History history[100];
int hist_count = 0;

void add_history(const char *op, double a, double b, double res, int is_int) {
    if (hist_count < 100) {
        strcpy(history[hist_count].op, op);
        history[hist_count].a = a;
        history[hist_count].b = b;
        history[hist_count].result = res;
        history[hist_count].is_int = is_int;
        hist_count++;
    }
}

void print_history() {
    int i;
    printf("\n--- History ---\n");
    for (i = 0; i < hist_count; i++) {
        if (history[i].is_int) {
            printf("%d %s %d = %d\n", (int)history[i].a, history[i].op,
                (int)history[i].b, (int)history[i].result);
        } else {
            printf("%.2f %s %.2f = %.2f\n", history[i].a, history[i].op,
                history[i].b, history[i].result);
        }
    }
}

int main() {
    double (*ops[14])(double, double) = {
        add, sub, mul, NULL, mod_op, power, gcd, lcm,
        max_op, min_op, avg, absdiff, sumsq, prodsq
    };

    const char *op_names[] = {
        "Add", "Subtract", "Multiply", "Divide", "Modulus", "Power",
        "GCD", "LCM", "Max", "Min", "Average", "AbsDiff", "SumSq", "ProdSq"
    };

    int choice, subchoice;
    double a, b, res;
    int is_int;

    while (1) {
        printf("\nMENU\n");
        for (int i = 0; i < 14; i++) {
            if (i == 3) printf("%2d. Divide (submenu)\n", i + 1);
            else printf("%2d. %s\n", i + 1, op_names[i]);
        }
        printf("15. Exit\n16. History\nChoice: ");
        scanf("%d", &choice);
```

```
        if (choice == 15) break;
        if (choice == 16) {
            print_history();
            continue;
        }
        if (choice < 1 || choice > 14) {
            printf("Invalid option.\n");
            continue;
        }

        if (choice == 4) {
            printf("Division submenu:\n1. Integer division\n2. Normal division\nSubchoice: ");
            scanf("%d", &subchoice);
            if (subchoice != 1 && subchoice != 2) {
                printf("Invalid subchoice.\n");
                continue;
            }
            printf("Enter two numbers: ");
            scanf("%lf %lf", &a, &b);

            if (subchoice == 1) {
                res = int_div(a, b);
                is_int = 1;
                add_history("IntDiv", a, b, res, is_int);
                printf("Result: %d\n", (int)res);
            } else {
                res = norm_div(a, b);
                is_int = 0;
                add_history("Div", a, b, res, is_int);
                printf("Result: %.2f\n", res);
            }
            continue;
        }

        printf("Enter two numbers: ");
        scanf("%lf %lf", &a, &b);
        res = ops[choice - 1](a, b);

        if (choice == 5 || choice == 7 || choice == 8) {
            is_int = 1;
        } else {
            is_int = 0;
        }

        add_history(op_names[choice - 1], a, b, res, is_int);

        if (is_int) printf("Result: %d\n", (int)res);
        else printf("Result: %.2f\n", res);
    }
    return 0;
}
```

(02)

```
#include <stdio.h>
#include <stdlib.h>

#define MEM_SIZE 200

char memory[MEM_SIZE];
int allocated[MEM_SIZE] = {0};

typedef struct {
    int start;
    int size;
    int used;
} AllocInfo;

AllocInfo allocs[MEM_SIZE];
int numAllocs = 0;


void* NewMalloc(int size) {
    if (size <= 0) return NULL;
    int count = 0, start = -1;

    for (int i = 0; i < MEM_SIZE; i++) {
        if (allocated[i] == 0) {
            if (start == -1) start = i;
            count++;
            if (count == size) {

                for (int j = start; j < start + size; j++) allocated[j] = 1;


                allocs[numAllocs].start = start;
                allocs[numAllocs].size = size;
                allocs[numAllocs].used = 1;
                numAllocs++;

                return (void*)&memory[start];
            }
        } else {
            start = -1;
            count = 0;
        }
    }
    return NULL;
}


void NewFree(void *ptr) {
    if (ptr == NULL) return;
    int addr = (char*)ptr - memory;
    if (addr < 0 || addr >= MEM_SIZE) return;

    for (int i = 0; i < numAllocs; i++) {
        if (allocs[i].used && allocs[i].start == addr) {
            for (int j = allocs[i].start; j < allocs[i].start + allocs[i].size; j++) {
                allocated[j] = 0;
            }
            allocs[i].used = 0;
            return;
```

```
        }
    }
}


void defrag() {
    AllocInfo temp[MEM_SIZE];
    int count = 0;

    for (int i = 0; i < numAllocs; i++) {
        if (allocs[i].used) {
            temp[count++] = allocs[i];
        }
    }

    int pos = 0;
    for (int i = 0; i < count; i++) {
        int start = temp[i].start;
        int size = temp[i].size;

        if (pos != start) {

            for (int j = 0; j < size; j++) {
                memory[pos + j] = memory[start + j];
            }

            for (int k = 0; k < numAllocs; k++) {
                if (allocs[k].used && allocs[k].start == start) {
                    allocs[k].start = pos;
                    break;
                }
            }
        }
        pos += size;
    }

    for (int i = 0; i < MEM_SIZE; i++) {
        allocated[i] = (i < pos) ? 1 : 0;
    }
}

void print_allocated() {
    printf("Allocated blocks:\n");
    for (int i = 0; i < numAllocs; i++) {
        if (allocs[i].used) {
            printf("  Start: %d, Size: %d bytes\n", allocs[i].start, allocs[i].size);
        }
    }
}

int main() {

    int *pInt = (int*) NewMalloc(sizeof(int));
    if (pInt != NULL) *pInt = 42;

    char *pChar = (char*) NewMalloc(sizeof(char));
    if (pChar != NULL) *pChar = 'A';

    double *pDouble = (double*) NewMalloc(sizeof(double));
    if (pDouble != NULL) *pDouble = 3.14159;
```

```c
    int *pArr = (int*) NewMalloc(5 * sizeof(int));
    if (pArr != NULL) {
        for (int i = 0; i < 5; i++) pArr[i] = (i + 1) * 10;
    }

    printf("Initial Allocations\n");
    print_allocated();


    NewFree(pChar);
    printf("\nAfter Freeing Char Variable\n");
    print_allocated();

    defrag();
    printf("\nAfter Defragmenting Memory\n");
    print_allocated();


    printf("\nStored Values\n");
    printf("int: %d, double: %.5f, array: ", *pInt, *pDouble);
    for (int i = 0; i < 5; i++) printf("%d ", pArr[i]);
    printf("\n");

    return 0;
}
```

```
ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc ques2.c

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Initial Allocations
Allocated blocks:
  Start: 0, Size: 4 bytes
  Start: 4, Size: 1 bytes
  Start: 5, Size: 8 bytes
  Start: 13, Size: 20 bytes

After Freeing Char Variable
Allocated blocks:
  Start: 0, Size: 4 bytes
  Start: 5, Size: 8 bytes
  Start: 13, Size: 20 bytes

After Defragmenting Memory
Allocated blocks:
  Start: 0, Size: 4 bytes
  Start: 4, Size: 8 bytes
  Start: 12, Size: 20 bytes

Stored Values
int: 42, double: 0.00000, array: 335544320 503316480 671088640 838860800 0
```

(03)

```c
#include <stdio.h>
#include <math.h>


typedef struct {
    double x;
    double y;
} Point;


double euclidean_distance(Point p1, Point p2) {
    double dx = p1.x - p2.x;
    double dy = p1.y - p2.y;
    return sqrt(dx * dx + dy * dy);
}

double manhattan_distance(Point p1, Point p2) {
    double dx = fabs(p1.x - p2.x);
    double dy = fabs(p1.y - p2.y);
    return dx + dy;
}

double chebyshev_distance(Point p1, Point p2) {
    double dx = fabs(p1.x - p2.x);
    double dy = fabs(p1.y - p2.y);
    if(dx > dy) {
        return dx;
    } else {
        return dy;
    }

}

int main() {
    Point p1 = {1.0, 2.0};
    Point p2 = {4.0, 6.0};

    printf("Point 1: (%.1f, %.1f)\n", p1.x, p1.y);
    printf("Point 2: (%.1f, %.1f)\n\n", p2.x, p2.y);

    printf("Euclidean Distance: %.2f\n", euclidean_distance(p1, p2));
    printf("Manhattan Distance: %.2f\n", manhattan_distance(p1, p2));
    printf("Chebyshev Distance: %.2f\n", chebyshev_distance(p1, p2));

    return 0;
}
```

```
ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc que3.c

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Point 1: (1.0, 2.0)
Point 2: (4.0, 6.0)

Euclidean Distance: 5.00
Manhattan Distance: 7.00
Chebyshev Distance: 4.00
```